Selenium:

- **Definition:** It is an open source automation tool which provide the functionality of automating the web based/mobile based application by using different programming language.
- Testing an application or software with the help of any automation tool and executing test case is called as automation.
- For web base application the most popular and easy to use open source tool is Selenium:
  Java + Selenium === Automation tool

Advantages of Selenium:

- Selenium is an **open source tool**
- Selenium supports various programming language to write the test scripts such as **Java, ruby, python, c++ etc.**
- Selenium supports various operating system such as **mac, Linux, Windows etc.**
- Selenium supports multiple browser like **Mozilla Firefox, Google chrome, Opera, Safari etc.**
- Selenium supports parallel test case execution.
- Selenium uses less hardware resources.

Disadvantages of Selenium:

- Selenium **does not automate captcha and barcode, image and video and audio**.
- Selenium only works on the **web based applications**.
- Difficult to use, takes more time to create test scripts.
- Selenium depends on third party frameworks like ***TestNG, Cucumber*** *for the reporting.*
- New features may not work properly.
- Selenium has no built-in reporting facility.

Advantages of Automation Testing:

- **Reusability of test scripts:** We did not write the test cases multiple times for same functionality so reusability is possible.
- **Project duration reduces:** So everything depends on delivery we can reduce the project duration by using agile methodology with automation.
- **Compatibility testing is easy:** We can execute it parallelly in automation testing.
- Required less human efforts.
- Using automation accuracy is more.

Disadvantage of Automation:

- Required skilled labor.
- More money investment required than manual.
- Automation tool maintenance required.

Automation testing tools:

- Automation testing is used to change the manual test case into a test script with the help of some automation tool.
- There are several types of automation testing tools available in the market.
    1. Selenium
    2. Watir
    3. QTP
    4. Telerik Studio
    5. Testins
    6. Applitools
    7. Cucumber

Selenium Components:

- Selenium is totally abstract i.e. we can say that it is an interface.
- **List of Selenium Components:**

  Selenium Remote Control (R C)
  Selenium Grid
  Selenium Integrated Development Environment (IDE)
  WebDriver

  - **Selenium RC: (Selenium Remote Control):**
    - Selenium remote control is officially deprecated by selenium and it used to work on JavaScript to automate the web applications.
    - It supported multiple browsers such as **Chrome, Firefox, Edge, Safari**, etc.
    - **Record and playback functionality was not supported.**
    - It required **starting the Selenium RC Server** before executing test scripts.
    - Its **core engine was JavaScript-based**.
    - It had a **small and simple API**, but APIs were **less object-oriented** compared to Selenium WebDriver.
    - It **did not support listeners**.
    - It **did not support testing mobile applications** such as **iPhone or Android apps**.
    - **Why Selenium RC Does Not Support Listeners :**
    - **Client–Server Architecture:**
    - Selenium RC was built on a client–server architecture, where the RC Server injected JavaScript into the browser to control it.
    - **Server (Selenium RC Server) :**
    - A Java-based component that had to be started before running tests.
    - Acted as a proxy server between the test scripts and the browser.
    - Injected JavaScript (Selenium Core) into the browser to simulate actions like:
        1. Click
        2. Type
        3. Navigate

- **Client (Test Scripts) :** Test scripts were written in languages like:
    1. **Java**
    2. **Ruby**
    3. **Python**

- These scripts used Selenium RC libraries and sent commands as HTTP requests to the RC Server.
- Example command: selenium.click("id=loginBtn");

- **Browser**
    1. The RC Server received commands from the client.
    2. It injected JavaScript into the browser to perform the action.
    3. The result was then returned back to the client script.

Selenium Grid:

- Selenium Grid is a component of the Selenium suite that allows testers to run test cases on multiple machines, browsers, and operating systems simultaneously.
- Its main purpose is to execute tests in parallel and distribute test execution across different environments.
- **Key Features of Selenium Grid :**

1. **Parallel Test Execution :**
    - Selenium Grid allows multiple test cases to run simultaneously on different machines. This significantly reduces test execution time.
2. **Cross-Browser Testing :**

    Tests can run on different browsers such as:
    - Google Chrome
    - Mozilla Firefox
    - Microsoft Edge
    - Safari
3. **Cross-Platform Testing :**

    Selenium Grid allows tests to run on different operating systems like:
    - Windows
    - Linux
    - macOS
4. **Distributed Test Execution :**
    - Tests can be executed on remote machines instead of only on the local system.

- **Selenium Grid Architecture :**

  Selenium Grid works on a Hub–Node architecture.

  1. Hub :
      - The Hub is the central server that controls the entire grid.

- **Responsibilities of Hub :**

  Receives test requests from the test scripts.

  Decides which node should execute the test.

  Routes the test commands to the appropriate node.

  Only one hub exists in a Selenium Grid setup.

  It acts as the central point of communication.

  Example: Test Script → Hub → Node

2. **Node :**
   - A Node is a machine connected to the hub that executes test cases.
   - Characteristics of Node:
   - A node can be:
     1. A physical machine
     2. A virtual machine
     3. A cloud machine
   - Each node can have:
     1. Different browsers
     2. Different operating systems
   - Node Responsibilities :
     1. Receives commands from the hub.
     2. Executes tests using Selenium WebDriver.
     3. Sends results back to the hub.

How Selenium Grid Works:

- The tester writes automation scripts using Selenium WebDriver.
- The script sends a request to the Hub.
- The Hub checks available nodes.
- The Hub selects the appropriate node based on:
  1. Browser
  2. Operating system
  3. Availability
- The Hub sends the test request to the selected node.
- The Node executes the test using WebDriver.
- The Node sends the result back to the Hub.

Communication in Selenium Grid:

Nodes receive requests from the hub in the form of JSON test commands.
These commands are executed by WebDriver on the node machine.
Example Flow:
Test Script
↓
Hub (Central Server)
↓
Node (Remote Machine)
↓
Browser Execution

Types of Selenium Grid Execution:

1. Hub and Node on Same Machine
    - Hub and node run on the same system.
    - Used mainly for learning or small projects.

2. Hub and Node on Different Machines
    - Hub runs on one machine.
    - Nodes run on multiple remote machines.
    - Used in large test environments.

Advantages of Selenium Grid:

1. Faster execution due to parallel testing.
2. Cross-browser testing support.
3. Cross-platform testing capability.
4. Scalability – multiple nodes can be added.
5. Better resource utilization.

- **Selenium Integrated Development Environment (IDE):**

- Selenium IDE is a Firefox plugin that works on record and playback principle.
- **It supports for record and playback.**
- **Doesn't required to start server before executing the test script.**
- It is a GUI Plug-in.
- Core engine is JavaScript based.
- It only works in Mozilla Firefox browser.
- It is not object oriented.
- It does not supports listeners.
- It does not support to test iPhone/Android applications.
- **Why Selenium IDE Does Not Support Listeners :**
- **Selenium IDE** also does **not support listeners** because:
    1. It is mainly a record-and-playback automation tool.
    2. It is not a full programming framework.
    3. It does not allow programmatic hooks into test execution.
    4. You cannot attach event handlers like in Selenium WebDriver.

- Reporting in Selenium IDE is limited to:
    1. Basic **log messages**
    2. **Exported test results**

- **What Are Listeners?**
    1. In Selenium WebDriver, listeners are special interfaces used to monitor test execution events.
    2. Common Use Cases
        a. Logging test start, success, failure, or skip
        b. Capturing screenshots on test failure
        c. Generating custom reports

3. Examples in Java
    - ITestListener (from TestNG)
    - WebDriverEventListener (from Selenium WebDriver)

WebDriver:

- Selenium WebDriver is a programming interface used to automate web-based applications using browser native methods.
- It directly communicates with the browser driver, without requiring an intermediate server.
- It is implemented through browser-specific drivers such as ChromeDriver, GeckoDriver, etc.
- It controls the browser by directly communicating with it.
- It supports multiple browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.
- It supports multiple programming languages such as: Java, Python, C#, Ruby etc.
- It supports the implementation of listeners through testing frameworks.
- It supports mobile testing for Android and iOS applications when integrated with tools like Appium.
- It does not support record and playback functionality.
- It does not require starting a server before executing the test script.
- **Architecture of Selenium WebDriver:**
  Selenium WebDriver uses a browser-specific driver to control the browser.
  Flow: Test Script → WebDriver API → Browser Driver → Browser
  Components:
    1. **Test Script:** Automation code written in supported languages like Java, Python, or C# etc.
    2. **WebDriver API:** Sends commands from the test script to the browser driver.
    3. **Browser Driver:** Translates WebDriver commands into browser-specific actions. Example: ChromeDriver, GeckoDriver, etc.
    4. **Browser:** Executes the actions (click, type, navigation) and returns responses to the test script.

- **Key Features of Selenium WebDriver:**
    1. **Supports Multiple Browsers:** Chrome, Firefox, Edge, Safari, and Opera.
    2. **Supports Multiple Operating Systems:** Windows, macOS, Linux.
    3. **No Intermediate Server:** Direct communication with the browser driver.
    4. **Handles Dynamic Web Elements:** Supports modern web applications built with AJAX.

5. **Advanced User Interactions**: Supports actions like Mouse hover, Drag and drop, Double-click etc.
6. **Supports Multiple Locators**: ID, Name, Class Name, Xpath, CSS Selector, Link Text, Partial Link Text, Tag Name etc.
7. **Integration with Frameworks:** TestNG, Junit, Maven, Jenkins, Reporting tools etc.

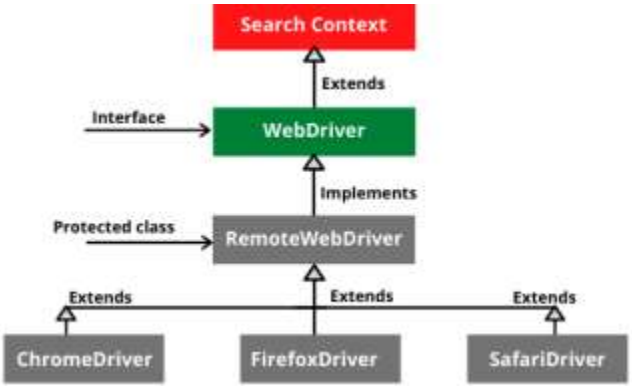
Selenium Architecture:

1. **SearchContext** is the top-most interface in Selenium which contains abstract method and inherited to WebDriver interface.
2. **WebDriver** interface extends **SearchContext**.
3. WebDriver is an interface which contain abstract method of SearchContext and its own abstract method.
4. **RemoteWebDriver** class implements **WebDriver**.
5. All the abstract methods are implemented **in RemoteWebdriver** class.
6. **RemoteWebdriver** is as class which implements all the abstract methods of both interfaces.
7. Browser-specific drivers like ChromeDriver, FirefoxDriver extend **RemoteWebDriver**.
8. To achieve this we need to perform **up casting** in selenium.

WebDriver driver = new ChromeDriver();

Class Hierarchy in Selenium:

```
java.lang.Object
  ↳ org.openqa.selenium.SearchContext (interface)
    ↳ org.openqa.selenium.WebDriver (interface)
      ↳ org.openqa.selenium.remote.RemoteWebDriver (class)
          ↳ org.openqa.selenium.chrome.ChromeDriver
          ↳ org.openqa.selenium.firefox.FirefoxDriver
          ↳ org.openqa.selenium.edge.EdgeDriver
          ↳ org.openqa.selenium.safari.SafariDriver
```



- **Search Context:**
  - SearchContext is the super-most interface in Selenium.
  - It provides methods to locate web elements.

- It is the parent interface of WebDriver
- All methods of SearchContext and WebDriver are implemented in RemoteWebDriver class.
- It contains two methods :

```
WebElement findElement(By by);
List<WebElement> findElements(By by);
```

findElement () - Returns the first matching web element
findElements () - Returns a list of matching elements

Selenium WebDriver :

- WebDriver is the main interface used to control the browser.
- Selenium WebDriver is an interface that defines **a set of methods.**
- The object of the web driver is a **browser.**
- All browser driver classes implement **WebDriver** indirectly through **RemoteWebDriver**.
- However, implementation is provided by the browser specific classes.
- Some of the implementation classes are Chrome Driver, Firefox Driver, and Safari Driver etc.
- For example :

  get(String url) - Opens a URL
  getTitle() - Returns page title

Selenium remote webdriver:

- **RemoteWebDriver** is the core implementation class of Selenium.
- It implements WebDriver interface.
- It acts as a bridge between WebDriver interface and browser drivers.
- It sends commands to browser drivers using W3C WebDriver Protocol.
- All browser-specific drivers extend RemoteWebDriver.

RemoteWebDriver
↑
ChromeDriver
↑
FirefoxDriver
↑
EdgeDriver

Browser-Specific Drivers:

These are concrete classes used to interact with real browsers.
Each driver communicates with the browser executable.
Executes commands sent by RemoteWebDriver.
For Example :

Driver: ChromeDriver **Browser**: Google Chrome
Driver: FirefoxDriver **Browser**: Mozilla Firefox
Driver: EdgeDriver **Browser**: Microsoft Edge

Selenium WebDriver Architecture Flow:

Test Script (Java / Python / C#)
|
WebDriver Interface
|
RemoteWebDriver (Core Implementation)
|
Browser Driver (ChromeDriver / FirefoxDriver)
|
Real Browser

Step-by-Step Flow:

- The test script calls WebDriver methods.
- The WebDriver interface forwards commands to RemoteWebDriver.
- RemoteWebDriver converts commands into browser-understandable format.
- The browser driver communicates with the actual browser.
- The browser executes the action.
- The result is returned back to the test script.

WebDriver Commands:

- **Get Method**
  - To launch URL in browser.
  - It is used to enter URL in a browser and it will wait for fully loaded the webpage.

    Syntax :- driver.get("url");
    driver.get("https://www.facebook.com/");
- **Get Title Method**
  - Returns title of page
  - It is used to get/return title of webpage

    Syntax :- driver.getTitle();
    driver.getTitle ();
- **Get CurrentURL Method**
  - Returns current URL of webpage
  - It is used to get/return current URL of webpage

    Syntax: - driver.getCurrentUrl ();
    driver.getCurrentUrl ();
- **Get PageSource Method**
  - **Returns HTML code of webpage**
    - **Syntax :-** driver.getPageSource();
    - driver.getPageSource();

- **Get Text Method:**
  - getText () command is used to retrieve *the inner text of the specified web element.*
  - The command doesn't *require any parameter and returns a* ***string value.***
    - String Text = driver.findElement(By.id("Text")).**getText();**

- ***Navigate Commands/Methods:***

  Navigate is used to back, forward, refresh the browsers.

  - **navigate().to()**
    - It is used to navigate from one webpage to other webpage and load
    - **new URL/webpage in the existing window of browser**

      Syntax: - driver.navigate().to("URL");
  - **Forward command**
    - This method is used to click on the forward button (arrow) of the browser window.

      Syntax:-driver.navigate ().forward ();
  - **Back command**
    - This method is used to click on back button(arrow) of the browser window

      Syntax: - driver.navigate ().back ();
  - **Refresh command**
    - This method is used to refresh/reloads the current webpage in the browser

      Syntax: - driver.navigate().refresh ();

- ***Browser Commands/Methods***
  - **Close:**
    - To close particular window
    - **This method is used to close current browser window or selenium focused browser window.**

      Syntax: - driver. Close();

  - **Quit**
    - To close all windows.
    - This method is used to quit/close all windows present in browser. It will use to end selenium script.

      Syntax :- driver.quit();

  - **Maximize**
    - To maximize launched browser.
    - This method is used to maximize the browser window.

      Syntax :- driver.manage().window().maximize();

Locators in Selenium:

DOM stands for **Document Object Model.**

In simple words, **DOM specifies the structural representation of HTML elements**.

Selenium Locators:

- Locators are nothing but it is technique used to find web elements which are present on webpage.
- Each web element has its certain position on webpage so selenium locators are used to find web element from webpage.
- Locators are methods of "**By** "class by which we can find element.
- The locators are one of the most important parameters for scripting base foundation and they may lead script failure.
- Locating elements in Selenium WebDriver is performed with the help of **findElement** () and **findElements** () methods provided by searchContext interface.
- There are 8 locators which are used to find the web elements from the webpage.

  ID
  Name
  Id
  TagName
  ClassName
  CSS selector
  Link Text
  Partial LinkText
  Xpath

  - **ID:**
    - Uses the id attribute of an element.
    - Fastest and most reliable if IDs are unique.
    - Example:

      Syntax: driver.findElement(By.id ("id value"));

  - **Name :**
    - Uses the name attribute of an element.
    - Useful if the name is unique.
    - Example:

      Syntax : driver.findElement(By.name("name_value"));

  - **className**
    - Locates elements using the class attribute.
    - Example:

      Syntax : driver.findElement(By.className("value"));

- **tagName:**
  - Locates elements by their HTML tag (e.g., <input>, <a>, <button>).
  - A tagName is a part of a DOM structure where every element on a page is been defined via tag like input tag, button tag or anchor tag etc.
  - Mostly used for finding multiple elements.

    Syntax: driver.findElement(By.tagName ());

- **linkText:**
  - Locates anchor tags (<a>) using the exact link text.
  - Elements can be located via link text that is present in the hyperlinks.
  - it start with <a>tag
  - Example:

    Syntax : driver.findElement(By.linkText("Text"));

- **partialLinkText**
  - Similar to Link Text, but matches a part of the link text.
  - Condition is - partial link text will be in sequential manner as shown below.
  - Example :

    Syntax: driver.findElement(By.partiallinkText("partial_text"));

- **CSS selector**
  - Very powerful and flexible.
  - Locates elements using CSS path**.**
  - CSS Selector syntax is quite similar to the XPath syntax.
  - It can be represented syntactically as follows:

    Syntax : node[attribute_name = 'attribute_value']
  - Node is the tag name of the HTML element, which needs to locate.
  - **attribute_name** is the name of the attribute which can locate the element.
  - **attribute_value** is the value of the attribute, which can locate the element.

Types of CSS Selectors:

1. **By ID (#id):**
   Select element by unique id.
   For ID attribute by using **#atttribute** value we can locate the element
   Example:
   driver.findElement(By.cssSelector("#username")).sendKeys ("testuser");
   Equivalent XPath: //input[@id='username']

2. **By Class (.class) :**
   Select element by class name.
   Example :
   driver.findElement(By.cssSelector(".login-btn")).click();
   Equivalent XPath: //button[@class='login-btn']

3. **By Tag and Class:**
   Finds a <button> element with class login-btn.
   Example:
   driver.findElement (By.cssSelector("button.login-btn")).click();

4. **By Attribute :**
   Example :
   driver.findElement(By.cssSelector("input[name='email']")).sendKeys ("abc@test.com");
   Equivalent Xpath: //input[@name='email']

5. **By Multiple Attributes:**
   Example:
   driver.findElement(By.cssSelector("input[type='text'][name='email']")).sendKeys("abc@test.com");
   Equivalent Xpath: //input[@type='text' and @name='email']

6. **By Tag and Attribute :**
   Example:
   driver.findElement(By.cssSelector("input[id='username']")).sendKeys ("testuser");

7. **Starts With (^=):**
   Matches attribute values starting with given text.
   Example:
   driver.findElement(By.cssSelector("input[id^='user']")).sendKeys ("testuser");
   Matches: id="user123"

8. **Ends With ($=):**
   Matches attribute values ending with given text.
   Example:
   driver.findElement(By.cssSelector("input[id$='name']")).sendKeys("abc");
   Matches: id="username"

9. **Contains (*=):**
   Matches attribute values containing given text.
   Example:

driver.findElement(By.cssSelector("input[id*='ser']")).sendKeys("abc");
Matches: id="username"

CSS Position Selectors :

10. **First Child:**
    Example: **driver.findElement(By.cssSelector("ul li:first-child"));**

11. **Last Child:**
    Example: driver.findElement(By.cssSelector("ul li:last-child"));

12. **Nth Child:**
    Example: driver.findElement(By.cssSelector("ul li:nth-child(3)"));

CSS Relationship Selectors:

13. **Descendant Selector (space):**
    Selects elements inside another element.
    Example: driver.findElement(By.cssSelector("div.form input"));
    Finds <input> inside .

14. **Direct Child Selector (>):**
    Selects direct child only.
    Example: driver.findElement(By.cssSelector("div.form > input"));

15. **Sibling Selector:**
    Immediate Next Sibling (+)
    Example: driver.findElement(By.cssSelector("label + input"));

16. **All Following Siblings (~):**
    Example: driver.findElement(By.cssSelector("label ~ input"));

- **Xpath**:
  - XPath is the most powerful locator in Selenium.
  - Can navigate forward and backward in the DOM.
  - Xpath or XML path is a query language for selecting the nodes from XML documents.
  - Useful when IDs or classes are not available.
  - **Example :**
    - driver.findElement(By.xpath("//input[@id='username']")).sendKeys("test user");

- **Types of X-path :**
    - Absolute Xpath
    - Relative Xpath

- **Absolute Xpath:**
    - **Definition:** Absolute XPath is the direct path from the root node (html) to the desired element in the HTML DOM structure.
    - It starts from the root node (html).
    - It uses a single forward slash / to define the path.
    - It follows the complete hierarchy of elements from parent to child.
    - In browsers, **"Copy Full XPath"** usually generates an **Absolute XPath**.
    - **Syntax:** /tagname/tagname/tagname
    - **Example :** /html/body/div[1]/div[2]/form/input
    - **Explanation :**

      The path starts from the html root node.

      It then moves step-by-step through body → div → form → input.
    - **Characteristics:**

      Traverses the entire HTML DOM structure.

      Moves from parent to immediate child elements.
    - **Disadvantages of Absolute Xpath:**

      If any change occurs in the HTML structure, the XPath will break.

      It becomes very long and complex.

      It is not reliable for dynamic web pages.

- **Relative Xpath:**
    - **Definition:** Relative XPath is used to locate elements by starting from anywhere in the HTML DOM instead of the root node.
    - It uses double forward slashes //.
    - It searches for elements anywhere in the webpage.
    - It is shorter and more flexible.
    - In browsers, **"Copy XPath"** usually generates a **Relative XPath**.
    - **Syntax :** //tagname[@attribute='value']
    - **Example:** //input[@id='username']
    - **Explanation :**

      // means search anywhere in the document.

      input is the tag name.

      @id='username' is the attribute used to identify the element.
    - **Characteristics:**

      Does not require the complete path.

      Can start from the middle of the DOM structure.

      Can skip intermediate elements.

- **Advantages of Relative Xpath:**
  Shorter and easier to write.
  More stable when page structure changes.
  Highly preferred in Selenium automation.

1. **Xpath by using attribute name.**
   - We can use different attributes and its values and create Xpath.
   - **Syntax :** //tagname[@attribute = ‘value’];
   - Eg: //input[@id = 'lp123'];

2. **XPath with Multiple Attributes.**
   - Combines multiple conditions..
   - **Syntax :** //tagname[@attribute = ‘value’ and [@attribute = ‘value’];
   - Example : //input[@type='text' and @name='email']

3. **Xpath by using text function:**
   - If we want to find element which is present along with link.
   - **Syntax :** //tagname[text() = ‘value’]
   - Eg: //a[text () = 'Forgot Password?']

4. **Xpath by using contains function.**
   - **By attribute**
     - If there are some characters of attribute which gets change dynamically (with respect to time) then it is prefer to use contains in the Xpath which provides the flexibility to create the Xpath by using some of the characters.
     - **Syntax :** //tagname[contains(@ attribute, “value”)]
     - Eg: //input[contains(@id, '567')]
   - **By text function**
     - **Syntax :** //tagname[contains(text(),”value”)]
     - Eg : //a[contains(text(),'often pass')]

5. **XPath with starts-with() :**
   Matches starting value of an attribute.
   Example: //input[starts-with(@id,'user')]

6. **XPath with ends-with():**
   Matches ending value of an attribute.
   Example://input[ends-with(@id,'user')]

7. **Xpath by index**

   When we write an Xpath and if duplicates occur.
   Whenever we have multiples nodes matching and we don't have any other options for the other attributes.
   Then we can use index and give numbers to it.
   Syntax : (whole Xpath expression)[number]
   Example : (//input[@name = 'Login'])[2]

8. **XPath with AND / OR:**

   Combines multiple conditions.
   Example:

   //input[@id='username' and @type='text']
   //input[@id='username' or @name='user']

9. **XPath with Axes (Advanced):**

   a. **parent :**
      //input[@id='username']/parent::div

   b. **child :**
      //div[@class='form-group']/child::input

   c. **following-sibling:**
      //label [text()='Email']/following-sibling:: input

   d. **preceding-sibling:**
      //input[@id='password']/preceding-sibling::label

   e. **ancestor:**
      //input[@id='username']/ancestor::form

   f. **descendant :**
      //form[@id='loginForm']/descendant::input

10. . **XPath with last()**

    Selects the last element.
    Example: (//input[@type='text'])[last()]

11. **XPath with position()**
    Selects element based on position.
    Example: (//button [@type='submit']) [position () =1]

12. **XPath using Wildcard (*):**
    Matches any element.
    Example: //*[@id='username']

Dimension and Point Class:

1. **Dimension Class :**

Purpose:

Used to resize the browser window.
Helps in testing UI stability and responsiveness.
Useful when verifying whether element position/size remains consistent.

Import Statement:

```
import org.openqa.selenium.Dimension;
```

Creating Dimension Object:

```
Dimension d = new Dimension(width, height);
```

Dimension → Selenium class
width → Browser width
height → Browser height
d → Reference variable

Set Browser Size:

```
driver.manage().window().setSize(d);
```

Get Browser Size:

```
Dimension size = driver.manage().window().getSize();
System.out.println(size);
```

Returns size in width & height coordinates.

Example:

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.Dimension;
```

```
public class Demo {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();

        driver.get ("https://example.com");

        Dimension d = new Dimension(1024, 768);
        driver.manage().window().setSize(d);

        Dimension currentSize = driver.manage().window().getSize();
        System.out.println(currentSize);
    }
}
```

2. Point Class :

Purpose:

Used to change browser window position on the screen.
Helps test UI behavior at different screen positions.

Import Statement:

```
import org.openqa.selenium.Point;
```

Creating Point Object:

```
Point p = new Point(x, y);
```

Point → Selenium class
x → Horizontal position
y → Vertical position
p → Reference variable

Set Browser Position:

```
driver.manage().window().setPosition(p);
```

Get Browser Position:

```
Point position = driver.manage().window().getPosition();
System.out.println(position);
```

Returns position in **(x, y) coordinates**

Example:

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.Point;

public class Demo {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();

        driver.get("https://example.com");

        Point p = new Point(200, 100);
        driver.manage().window().setPosition(p);

        Point currentPosition = driver.manage().window().getPosition();
        System.out.println(currentPosition);
    }
}
```

Window Handling in Selenium:

Window in Selenium:

1. A window is the main webpage that opens when a user hits a URL or clicks on a link/web element.
2. This is called the Parent Window (or Main Window).
3. In automation, sometimes multiple windows/tabs open.
4. Selenium allows switching control between these windows to perform actions.
5. After completing actions, control should return to the parent window.

Parent Window vs Child Windows:

1. **Parent Window:**
   The main window where the user initially lands.
   Opened when URL is launched.
2. **Child Windows:**
   Any new window/tab opened from the parent window.
   Example: Clicking a link that opens a new tab.

Window Handle:

Definition: A Window Handle is a unique identifier (ID) for each browser window**.**
It is used to switch control between windows.

Each window has a unique ID.
Return type is String.
Required to handle multiple windows or popups.

Switch to another Window:

driver.switchTo().window(windowID);

Methods for Window Handling:

- **getWindowHandle():**
  Used to get the ID of the current (parent) window.
  Features:
  Returns a single window ID
  Return type: String
  Syntax:
  String parentWindow = driver.getWindowHandle();

- **getWindowHandles():**
  Used to get IDs of all opened windows..
  Features:
  Returns multiple window IDs
  Return type: Set<String>
  Syntax:
  String parentWindow = driver.getWindowHandles();

Example: Switching Between Windows:

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import java.util.Set;

public class WindowHandlingDemo {
    public static void main(String[] args) {

        WebDriver driver = new ChromeDriver();
        driver.get("https://example.com");

        // Get Parent Window ID
        String parentWindow = driver.getWindowHandle();

        // Get All Window IDs
```

```
        Set<String> allWindows = driver.getWindowHandles();

        // Switch to Child Window
        for (String window : allWindows) {
            if (!window.equals(parentWindow)) {
                driver.switchTo().window(window);
                System.out.println("Switched to Child Window");
            }
        }

        // Switch back to Parent Window
        driver.switchTo().window(parentWindow);
        System.out.println("Back to Parent Window");
    }
}
```

Screenshot in Selenium:

1. **Why Do We Need Screenshots?**
   - While testing an application, if defects are found, we need proof.
   - Screenshots help to show test results (Pass/Fail).
   - Useful to verify application behavior after execution.
   - Helps in debugging when test cases fail.
   - Acts as evidence for bug reporting.

2. **Capture Screenshot in Selenium:**

   Step 1: Import Required Packages:

```
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.OutputType;
import java.io.File;
import org.openqa.selenium.io.FileHandler;
```

   Step 2: Typecast Driver to TakesScreenshot

```
TakesScreenshot ts = (TakesScreenshot) driver;
```

   TakesScreenshot → Interface

   driver → WebDriver object

   Step 3: Capture Screenshot

```
File source = ts.getScreenshotAs(OutputType.FILE);
```

   Screenshot is stored in temporary memory (source file)

Step 4: Create Destination Path

```
File destination = new File("path/to/save/screenshot.png");
```

Step 5: Copy Screenshot to Destination

```
FileHandler.copy(source, destination);
```

Example:

```
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.OutputType;
import java.io.File;
import org.openqa.selenium.io.FileHandler;

  public class ScreenshotDemo {
   public static void main(String[] args) throws Exception {

      WebDriver driver = new ChromeDriver();
      driver.get("https://example.com");

      // Typecast driver
      TakesScreenshot ts = (TakesScreenshot) driver;

      // Capture screenshot
      File source = ts.getScreenshotAs(OutputType.FILE);

      // Destination path
      File destination = new File("C:\\Screenshots\\example.png");

      // Copy file
      FileHandler.copy(source, destination);

      System.out.println("Screenshot captured successfully");
   }
}
```

Iframe in Selenium Webdriver:

An iFrame (Inline Frame) is a webpage embedded inside another webpage.
It is basically an HTML document inside another HTML document.
Used to display content from external sources like:

Advertisements
Videos
Widgets

HTML Tag : <iframe src="url"></iframe>

Why iFrame Handling is Important:

Selenium cannot directly interact with elements inside an iframe.
You must switch control to the iframe first before performing actions.

How to identify an iFrame:

- **Method 1: Inspect Element**
  Right-click on the element → Inspect
  Look for <iframe> tag in DOM

- **Method 2: View Page Source**
  Right-click → View Page Source
  Search for iframe
  If <iframe> tag exists → page contains iframe

- **Method 3: Browser Option**
  Right-click on element
  If you see option like "This Frame" → it is an iframe

How to switch over the elements in iframes:

- **Switch to the frame by index:**
  Index is one of the attributes for frame handling in selenium through which we can switch to it.

```
driver.switchTo ().frame (0);
driver.switchTo ().frame (1);
```

- **Switch to the frame by Name or ID:**
  Name and ID are attributes for frame handling in Selenium through which we can switch to the iframe.

```
driver.switchTo ().frame ("iframe1");
driver.switchTo ().frame ("id of the element");
```

- **Switch to the frame by Web Element:**

  We can switch to the iframe using web element.

  driver.switchTo ().frame (WebElement);

- **How to switch back to the Main Page.**

  To move back to the parent frame

  driver.switchTo ().parentFrame ();

  To get back to the main (or most parent) page,

  driver.switchTo ().default Content ();

JavaScript Executor:

- JavaScriptExecutor is an interface used to execute JavaScript code through Selenium WebDriver.
- It is useful when Selenium actions (click, sendKeys, etc.) don't work properly.
- **Methods Provided:**

  executeScript() → Executes JavaScript synchronously

  executeAsyncScript() → Executes JavaScript asynchronously

Typecasting WebDriver:

JavascriptExecutor js = (JavascriptExecutor) driver;

Scroll down a page using JavaScript:

JavaScriptExecutor js = (JavaScriptExecutor) driver;

js.executeScript ("window.scrollBy (0, 1000)");

Scroll down a page up to Web Element:

JavascriptExecutor js = (JavascriptExecutor) driver;

js.executeScript ("arguments [0].scrollIntoView ();" , Element);

Alternative way to click on login button:

JavascriptExecutor js = (JavascriptExecutor) driver;

js.executeScript ("arguments [0].click ();" , button);

Alternative way to send keys:

JavascriptExecutor js = (JavascriptExecutor) driver;

WebElement username =

driver.findElement (By.*xpath* ("//input [@name='username']"));

js.executeScript ("arguments [0].value='Admin'", username);

Handling of List Box (Drop down):

A List Box (Dropdown) is a web element that contains multiple options.
The user can select one or more values from the list.
Usually created using the <select> tag in HTML.

Handling List Box using Select Class:

Step 1: Import Package:

```
import org.openqa.selenium.support.ui.Select;
```

Step 2: Identify Web Element:

```
WebElement element = driver.findElement(By.xpath("xpath"));
```

Step 3: Create Select Object:

```
Select s = new Select(element);
```

Methods to Select Value:

- **selectByIndex()**

```
s.selectByIndex(5);
```

- **selectByValue()**

```
s.selectByValue("5");
```

- **selectByVisibleText()**

```
s.selectByVisibleText("1997");
```

For example:

```
WebElement xyz= driver.findElement (By.xpath ("Xpath"));
Select s=new Select (xyz);
s.selectByIndex (5);
s.selectByValue ("5");
s.selectByVisibleText ("1997");
```

Count of options:

- We have methods getoptions () and size () which help us to find count options from particular list box
- Below statement gives total count of options.

```
Select s=new Select();
List all_options = s.getOptions();
int count=all_options.size();
SOP(count);
```

Print all Options from list box:

```
 for (int i=0; i<count; i++){
String s=all_options.get (i).getText();
SOPln(s);
}
```

2nd Way to handle dropdown: if select tagname is not present in DOM then we can handle the dropdown or listbox by using find elements Methods

```
List<WebElement> daylist = driver.findElements(By.xpath("(//*[@id='day'])//option"));

int count = daylist.size();

for (WebElement day : daylist) {

    System.out.println("The Daylist as:"+ " "+  day.getText() + " " );

     if (day.getText().equals("27")) {

        day.click();
        }
    }
```

Use of Actions Class in Selenium WebDriver

1. **What is Actions Class?**
   The Actions class is used to perform mouse and keyboard actions.
   It helps handle complex user interactions like:
   - Drag and Drop
   - Right Click
   - Double Click
   - Hover actions

   Import Package:
   import org.openqa.selenium.interactions.Actions;

   Create Actions Object:
   Actions a = new Actions(driver);

- **Drag and Drop:**
  - This is an action performed with a mouse when a user moves (drags) a web element and then places (drops) it into an alternate area.
  - **dragAndDrop(WebElementsource, WebElement target):**
  - This method performs left click, hold the click operations to hold the source element, moves to the location of the target element and then releases the mouse click.
  - *Syntax:*

```
Actions a = new Actions(driver);
WebElement source = driver.findElement(By.xpath("source_xpath"));
WebElement target = driver.findElement(By.xpath("target_xpath"));
a.dragAndDrop(source, target).perform();
```

- **clickAndHold() :**
  - **dragAndDrop ()** method first performs click-and-hold at the location of the source element.
  - **moveToElement ()** : Then source element gets moved to the location of the target element.
  - **release () :** Finally, it releases the mouse.
  - **build ()** : build () method is used to perform action on multiple webelement at a time means it help to combine all the webelement to perform action. The build is executed in perform method internally.
  - **perform () :** method used to perform single action on single click.
  - **Syntax:**

```
Actions a = new Actions (driver);
WebElement source = driver.findElement(By locator);
WebElement target = driver.findElement(By locator);
a.clickAndHold (source).moveToElement (target).release ().build ().perform ();
```

- **dragAndDropBy(WebElementsource, int xOffset, int yOffset)**

  - This method clicks & holds the source element and moves by a given offset, then releases the mouse. Offsets are defined by x & y.
  - xOffset is horizontal movement (pixel value)
  - yOffset is a vertical movement (pixel value)
  - ***Syntax :***

```
Actions a = new Actions (driver);
WebElement from = driver.findElement(By locator);
WebElement to = driver.findElement(By locator);
a.dragAndDropBy (from, xOffset, yOffset).release ().build ( ).perform ();
```

- **Some of Click operation also we can do by using Action Class:**
  - **click() :**
    - This method we used to single click on any webelement.
    - *Syntax :*

      Actions a = new Actions (driver);
      WebElement r = d.findElement(By.xpath("//a[@title='jQuery UI']"));
      a.click();

  - **doubleClick() :**
    - This method we used to double click on any webelement.
    - *Syntax :*

      Actions a = new Actions (driver);
      WebElement r = d.findElement(By.xpath("//a[@title='jQuery UI']"));
      a.doubleClick(r).perform ();

  - **contextClick() :**
    - This method we used to right click on any webelement.
    - *Syntax :*

      Actions a = new Actions (driver);
      WebElement r = d.findElement(By.xpath("//a[@title='jQuery UI']"));
      a.contextClick(r).perform();

Some of Keyword action also we can do by using action class:

- **For Autosuggestion:**
  - Below code here we have selected apple search 3rd option:

```
System.setProperty("webdriver.chrome.driver", "E:\\D_ChromeDriver\\chromedriver.exe");

            WebDriver driver = new ChromeDriver () ;

            driver.get("https://www.google.com/");

            driver.manage().window().maximize();

      WebElement textfield = driver.findElement(By.xpath("//*[@name='q']"));

                textfield.sendKeys("Apple");

                Thread.sleep(2000);

                Actions act = new Actions (driver);

        act.sendKeys (textfield, Keys.ARROW_DOWN).sendKeys (Keys.ARROW_DOWN).sendKeys
(Keys.ENTER).build ().perform ();
```

- **For copy and paste data from one field and to enter in other field:**

```
System.setProperty("webdriver.chrome.driver",
"E:\\D_ChromeDriver\\chromedriver.exe");

WebDriver driver = new ChromeDriver () ;

driver.get("https://mbasic.facebook.com/reg/?cid=103&refsrc=deprecated&_rdr");

driver.manage().window().maximize();

WebElement namefield =
driver.findElement(By.xpath("//*[@name='firstname']"));

Actions act = new Actions (driver);

//to select all

act.sendKeys(namefield,
"Tanvir").keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL).build().perform();

// to copy data from one field
act.keyDown(Keys.CONTROL).sendKeys("c").keyUp(Keys.CONTROL).build().perform();

WebElement lastname =
driver.findElement(By.xpath("//*[@name='lastname']"));

lastname.click();

// to paste data to other field
act.keyDown (Keys. CONTROL).sendKeys ("v").keyUp (Keys.CONTROL).build ().perform ();
```

- **Tab Actions execution by Action Class:**

```
System.setProperty("webdriver.chrome.driver", "E:\\D_ChromeDriver\\chromedriver.exe");

WebDriver driver = new ChromeDriver () ;

driver.get("https://mbasic.facebook.com/reg/?cid=103&refsrc=deprecated&_rdr");

driver.manage().window().maximize ();

driver.findElement(By.xpath("//input[@name='firstname']")).sendKeys("Tanvir");

Actions act = new Actions (driver);
act.sendKeys(Keys.TAB).sendKeys("Shinde").build().perform();

act.sendKeys(Keys.TAB).sendKeys("8208159093").build().perform();

WebElement gender = driver.findElement(By.xpath("//*[@value='2' and @id='sex']"));

act.sendKeys (gender, Keys.TAB).sendKeys (Keys.ENTER).build().perform ();
```

Pop-Up Handling in Selenium WebDriver:

What is a Pop-Up?
A popup is a small window that appears on a webpage to display information or ask for user input. It usually appears after performing some action like clicking a button, submitting a form, etc.

Types of Pop-Ups in Selenium:

1. **Hidden Division Pop-Up (HTML Popup):**

Definition:
Created using **HTML, CSS, and JavaScript.**
Also called **dialog / overlay.**

Key Points:
Initially hidden, appears after user action
Can be inspected using Inspect tool
Not a JavaScript alert
Cannot be dragged/moved like a window
May have scroll bar if content is large
Takes focus when opened, returns focus when closed

Examples:
Calendar popup
Contact form
Success/Error messages

Handling in Selenium:
Treated like normal web elements
No special handling required

```
driver.findElement(By.xpath("//input[@type='text']")).sendKeys("Mayur");
```

2. **Child Browser (Window) Pop-Up:**

Definition:
Opens in a new browser window/tab.

How to Identify:
Can be inspected
Has minimize/maximize/close buttons
Can be moved around
Looks like a separate browser window

Handling Steps:

1. **Get Parent Window ID:**

```
String parent = driver.getWindowHandle();
```

2. Get All Window IDs:

```
Set<String> allWindows = driver.getWindowHandles();
```

3. Switch to Child Window:

```
for(String win : allWindows){
 if(!win.equals(parent)){
    driver.switchTo().window(win);
 }
}
```

4. Switch Back to Parent Window:

```
driver.switchTo().window(parent);
```

3. Alert Pop-Up (JavaScript Alert):

Definition:

Small message box displayed by JavaScript
Cannot be inspected using Inspect tool.

Types of Alerts:

a) Simple Alert (OK Button):

Purpose:

Displays information only

Handle:

```
driver.switchTo().alert().accept();
```

b) Confirmation Alert (OK & Cancel)

Purpose:

Takes user confirmation.

Handle:

i. Accept:

```
driver.switchTo().alert().accept();
```

ii. Dismiss:

```
driver.switchTo().alert().dismiss();
```

iii. Get Text:

```
String text = driver.switchTo().alert().getText();
```

c) Prompt Alert (Input Required):

Purpose:

Takes input from user

Handle:

```
Alert alert = driver.switchTo().alert();
alert.sendKeys("12345");
alert.accept();
```

Synchronization (Selenium Waits):

What is Synchronization?

Matching Selenium script speed with application/browser speed is called Synchronization.
Required because web elements may take time to load.

What are Waits?

Waits are dynamic components that pause execution until a specific condition is met.
Once the condition is satisfied, execution continues immediately (no need to wait full time).

Types of Waits in Selenium:

- Implicit Wait
- Explicit Wait
- Fluent Wait

a. **Implicit Wait:**

Definition:

A global wait applied to all elements in the script.
WebDriver waits for a specified time before throwing NoSuchElementException.

Key Points:

Default value = 0 seconds
Applied once, works for all elements
Supports multiple time units (Seconds, Minutes, Hours)
Not condition-based (only waits for element presence)

Syntax:

```
driver.manage().timeouts().implicitlyWait(30, TimeUnit.SECONDS);
```

Behavior:

If element found before time → continues immediately
If not found → waits till timeout → throws exception

b. **Explicit Wait:**

Definition:

Waits for a specific condition to occur before proceeding.

Key Points:

Applied to specific element only
Condition-based wait
Stops immediately once condition is met
More flexible than implicit wait

Syntax:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement element = wait.until(
    ExpectedConditions.elementToBeClickable(By.id("button"))
);
```

Common Expected Conditions:

alertIsPresent()
elementToBeClickable()
elementToBeSelected()
presenceOfElementLocated()
visibilityOfElementLocated()
invisibilityOfElementLocated()
textToBePresentInElement()
visibilityOfAllElementsLocatedBy()
presenceOfAllElementsLocatedBy()

c. Fluent Wait :

Definition:

Waits for a specific condition to occur before proceeding.
Advanced version of Explicit Wait with: Custom polling frequency, Exception handling.

Key Features:

Defines maximum wait time
Defines polling interval (check every few seconds)
Can ignore exceptions like NoSuchElementException

Syntax:

```
FluentWait wait = new FluentWait (driver)
                    .withTimeout (50, TimeUnit.SECONDS)
                    .pollingEvery (9, TimeUnit.SECONDS)
                    .ignoring (NoSuchElementException. Class);

Wait. Until (ExpectedConditions.elementSelectionStateToBe (By.xpath ("//*[@type=
'check box']"), true));
```

Behavior:

Keeps checking condition at defined intervals

Stops when:

Condition is met

Timeout reached

Parameterization:

The process of fetching information from the external source and using it into selenium test script is called as *'Parameterization'*.

To perform the action of parameterization we need to import the third party tool that ***"Apache POI"***.

APECHE POI- Apache provides a very famous library POI, by using this library. We can read & write the **XLSX** file format **Excel**.

CLASSES TO HANDLE EXCEL-SHEET

To fetch the data from excel sheet we have to use the class.

FileInputStream f = new FileInputStream (path);

To export the data to the excel-sheet we have another class.

FileOutputStream j = new FileOutputStream (path);

Methods:

- **getStringCellValue (); -** it will fetch the string data from the excelsheet.
- **getNumericCellValue (); -** it will fetch only numeric value from the excelsheet.
- **getCellType(); -** it will help us to check which type of data stored in particular cell.
- **getLastCellNumber(); -** it will shows the last cell number in excel sheet.
- **getRow();**
- **getCell();**

DIFFERENT INTERFACES

Workbook- This interface has two implementation classes

- *HSSFWorkbook*- it represents .xls file
- *XSSFWorkbook*- it represents .xlsx file

Sheet- It is also an interface; it has to two implementation classes.

- *HSSFSheet*- it represents .xls file
- *XSSFSheet*- it represents .xlsx file

Row- It is also an interface; it has to implementation classes.

- *HSSFRow*- it represents .xls file
- *XSSFRow*- it represents .xlsx file

Cell- It is also an interface; it has to implementation classes.

- *HSSFCell*- it represents .xls file
- *XSSFCell*- it represents .xlsx file.

- **To fetch the data from excel sheet :**

```
public static  String readData (int row , int column)throws IOException  {

        String path =".\\TestData\\MavenProjectTestData.xlsx";

FileInputStream excelfile = new FileInputStream(path);

        //to load the workbook

        XSSFWorkbook workbook = new XSSFWorkbook(excelfile);

        // to get the Sheet from the workbook

        XSSFSheet sh1 = workbook.getSheetAt(0);

        String value = sh1.getRow(row).getCell(column).getStringCellValue();

        System.out.println(value);

        return value;

}
```

- **To Write the data into the excel sheet :**

```
      public static void writeData ( int row , int column , String value)throws
IOException {

            String path =".\\TestData\\MavenProjectTestData.xlsx";

        XSSFWorkbook workbook = new XSSFWorkbook();

       XSSFSheet sheet = workbook.createSheet(value);

        sheet.createRow(row).createCell(column).setCellValue(value);

            FileOutputStream fout = new FileOutputStream(path);

            workbook.write(fout);
      }

}
```

Questions on Basic Selenium:

What is Selenium Web driver?

- Selenium WebDriver is a browser automation framework that accepts commands and sends them to a browser.
- It is implemented through a browser-specific driver.

- It directly communicates with the browser and controls it.
- Selenium WebDriver supports various programming languages like – Java, C#, PHP, Python, Perl, Ruby. And JavaScript.

What is Selenium Grid and when do we go for it?

- Selenium Grid can be used to execute same or different test script on multi-platform and browsers concurrently so as to achieve distributed test execution.
- **We use Selenium Grid in the following scenarios:**
    - Execute your test on different operating systems
    - Execute your tests on different versions of same browser
    - Execute your tests on multiple browsers
    - Execute your tests in parallel and multiple threads.

What are the advantages of Selenium Grid?

- It allows multi browser testing.
- It allows us to execute test case on multi-platform.
- Reduces batch processing time.
- Can perform multi-OS testing.
- It allows running the test case in parallel there by saving test execution time.

When should I use Selenium IDE?

- Because of its simplicity, Selenium IDE should only be used as a prototyping tool, not an overall solution for developing and maintaining complex test suites.

What is a Node in Selenium Grid?

- Node is a remote device that consists of a native OS and a remote WebDriver.
- It receives requests from the hub in the form of JSON test commands and executes them using WebDriver.
- There can be one or more nodes in a grid.
- Nodes can be launched on multiple machines with different platforms and browsers.
- The machines running the nodes need not be the same platform as that of the hub.

What are the types of WebDriver API's that are supported/available in Selenium?

- Selenium Web driver supports most of the browser driver APIs like Chrome, Firefox, Safari and PhantomJS.

Which WebDriver implementation claims to be the fastest?

- The fastest implementation of web driver is the HTMLUnitDriver.
- It is because the HTMLUnitDriver does not execute test on browser.
- It is based on HtmlUnit.
- It is known as Headless browser driver.
- It is same as Chrome, Edge, Firefox driver, but it does not have GUI so we cannot see the test execution on screen.

Is Selenium Server needed to run Selenium WebDriver scripts?

- In case of selenium Webdriver, it does not require to start selenium server for executing test scripts.
- Selenium Webdriver makes the calls between browser & automation script.

How to Start a Hub and node in Selenium Grid?

- **Start the Hub**

  java -jar selenium-server-standalone-3.14.jar -role hub

- **Connect Node to the Hub at localhost:4444**

  java –jar selenium-server-standalone-3.14.jar –role node –hub

  http://hubIP:4444/grid/register

What is an alternative to driver.get () method to open a URL using Selenium WebDriver?

- We can use driver.navigate ().To ("URL") method to open a URL.

What is the difference between driver.get ("URL") and driver.navigate ().to ("URL") commands?

- driver.get () is used to navigate particular URL (website) and wait till page load.
- driver.navigate () is used to navigate to particular URL and does not wait to page load.
- It maintains browser history or cookies to navigate back or forward.

What are the different types of navigation commands in Selenium?

Different navigation commands in selenium are: –

- navigate ().to (); -
- Navigate ().forward (); -
- Navigate ().back (); -
- Navigate ().refresh ();

How to fetch the current page URL in Selenium WebDriver?

- We can use the method to get the current page URL.

How can we maximize browser window in Selenium WebDriver?

- We can use the maximize () method to maximize browser window.
- driver.manage ().window ().maximize ();

What is the difference between driver.getWindowHandle () and driver.getWinowHandles () in Selenium WebDriver and their return type?

- **driver.getWindowHandle()** – To get the window handle of the current window. Returns a string of alphanumeric window handle
- **driver.getWindowHandles()** – To get the window handle of all current windows. Return a set of window handles.

How to resize browser window using Selenium WebDriver?

- driver.manage ().window ().maximize ();

What are the different ways for refreshing the page using Selenium WebDriver?

- Browser refresh operation can be performed using the following ways in Selenium.
  - **Refresh method**
    driver.manage ().refresh ();

  - **Get method**
    driver.get (“https://www.google.com”);
    driver.get (driver. GetCurrentUrl ());

  - **Navigate method**
    driver.get (“https://www.google.com”);
    driver.navigate.to (driver. GetCurrentUrl ());

  - **Send Keys method**
    driver. findElement (By.id("username")).sendKeys (Keys.F5);

How to find more than one web element in the list?

- We can find more than one web element by using the findElements () method in Selenium.
- List elements = driver.findElements (By.tagName (“a”));

List some scenarios which we cannot automate using Selenium WebDriver?

- Automating captcha is not possible using selenium webdriver.
- We cannot read bar code using Selenium WebDriver.
- We cannot automate OTP submission.

What is XPath Axes and what are the different Axes available?

- XPath axes search different nodes in XML document from current context node.
- XPath Axes are the methods used to find dynamic elements.
- **Preceding:** Select all nodes that come before the current node.
- **Following:** Selects all elements that come after the current node.
- **ancestor:** The ancestor axis selects all ancestors element (grandparent, parent, etc.) of the current node.
- **descendant:** Selects the descendants (child or grandchild) of the current node.
- **child:** Selects all children elements of the current node.
- **parent:** Selects the parent element of the current node.
- **Preceding-sibling:** Select the **Preceding** siblings of the context node.
- **Following-sibling:** Select the following siblings of the context node.

What is the default timeout of Selenium WebDriver?

- Default timeout is 30 seconds

How to retrieve typed text from a text box?

- String text = driver.findElement(By.id("textBox")).getText ();

What is the major differences between XPath Expressions and CSS Selectors?

- Using XPath we can traverse both forward and backward whereas CSS selector only moves forward in HTML DOM.

Which package can be imported while working with WebDriver?

- Following package can be imported:
  org.openqa.selenium.WebDriver.

How to send alt or shift or control or enter or tab key in Selenium WebDriver?

- We can use sendKeys () method to send any key.
  driver.findElement(element).sendKeys(Keys.TAB);

What is HtmlUnitDriver?

- HTML UnitDriver is the most light weight and fastest implementation headless browser for of WebDriver. It is based on HtmlUnit.
- It is known as Headless Browser Driver.
- It is same as Chrome, IE, or Firefox driver, but it does not have GUI so one cannot see the test execution on screen.

What happens when I write WebDriver driver new WebDriver ()?

- **Explanation:** WebDriver is an interface that is available in Selenium jar files. *driver is webDriver reference variable.*
- New is a keyword, we use to create an instance of the class (browser class).

What happens if we write ChromeDriver driver new ChromeDriver?

- If you use ChromeDriver driver = new ChromeDriver();
- The ChromeDriver instance which will get created through that we will be only able to invoke and act on the methods implemented by ChromeDriver and supported by Chrome Browser only.

Which locator is claim to be fast Xpath and CSS Selector?

- CSS selector locator is claim to be fast because it is short.

How to clear the text inside the text box fields using Selenium WebDriver?

- Syntax: driver.findElement (By.Id (“textbox_id”)).clear ();

How to get an attribute value of an element using Selenium WebDriver?

- driver.findElement (By.Id (“button_id”)).get Attribute (“text”);

How to press Enter key on text box in Selenium WebDriver?

- driver.findElement (By.Id (“button_id”)).sendKeys (keys. ENTER);

How to pause a test execution for 5 seconds at a specific point?

- We can pause test execution for 5 seconds by using the wait command.
  Syntax: driver.wait (5);
  Thread.sleep (5000);

- **Is Selenium Server needed to run Selenium WebDriver scripts?**

- In case of Selenium WebDriver*, it does not require to start Selenium Server* for executing test scripts.
- Selenium WebDriver makes the calls *between browser & automation script.*

What happens if we run this command driver.get ("www.google.com")?

- It will load a new web page in the current browser window with the website URL set to "www.google.com".
- This is done using an *http get operation*, and the method will block until the load is complete.

What is meaning of this statement System.setProperty("webdriver.chrome.driver" ,''value'');

- **System**: System is a class belongs to the java. Lang package.
- **SetProperty**() : It is used to set properties for the desired browser that to be used in the test automation.
- ***The setProperty()methods has two attributes:***
- **PropertyName** : It represents the name of the browser specific driver.
- **Value**: value point to the path of that browser driver.

Cookies in Selenium Cookies:

- Cookies are the files created by websites which you visit and is sent to your browser.
- It helps the site to remember information about your visit, which makes online experience easy by saving browsing data.
- How can we delete cookies in selenium?
- **Syntax:** driver. manage ().deleteAllCookies ();

What are the different types of exceptions you have faced in Selenium WebDriver?

- Different types of exceptions in Selenium are: –
  1. NoSuchElementException.
  2. NoSuchWindowException.
  3. NoSuchFrameException.
  4. NoAlertPresentException.
  5. ElementNotVisibleException.
  6. ElementNotSelectableException.
  7. TimeoutException.

How to clear the text inside the text box fields using Selenium WebDriver?

- Syntax: driver.findElement (By.Id ("textbox_id")).clear ();

How to get an attribute value of an element using Selenium WebDriver?

- driver.findElement (By.Id (“button_id”)).get Attribute (“text”);

How to press Enter key on text box in Selenium WebDriver?

- driver.findElement (By.Id (“button_id”)).sendKeys (keys. ENTER);

How to delete cookies in Selenium?

- We can use deleteAllCookies () method to delete cookies in selenium.
  driver.manage ().deleteAllCookies ();

What is the difference between driver.getWindowHandle () and driver.getWinowHandles () in Selenium WebDriver and their return type?

- **driver.getWindowHandle() –**
  To get the window handle of the current window.
  Returns a string of alphanumeric window handle
- **driver.getWindowHandles() –**
  To get the window handle of all open windows.
  Return a set of window handles.

How to handle hidden elements in Selenium WebDriver?

- **We can use the JavaScriptExecutor to handle hidden elements.**
  JavascriptExecutor js = (JavascriptExecutor) driver;
  js.executeScript ("document.getElementById ('displayed-text').value='text123'");

How to find more than one web element in the list?

- We can find more than one web element by using the findElements () method in Selenium.
  List elements = driver.findElements (By.tagName (“a”));

What is JavaScript Executor and in which case JavascriptExecutor will help in Selenium automation?

- JavaScriptExecutor is an **Interface** that helps to execute JavaScript functions through Selenium Web driver.
- In case, when selenium locators do not work you can use JavaScriptExecutor.
- You can use JavaScriptExecutor to perform a desired operation on a web element.

How to scroll web page up and down using Selenium WebDriver?

- To scroll using Selenium, you can use JavaScriptExecutor interface that helps to execute JavaScript methods through Selenium Webdriver.
  JavascriptExecutor js = (JavascriptExecutor) driver;
  js.executeScript ("arguments [0].scrollIntoView ();" , Element);

- **What is the alternative way to click on login button?**

- We can use JavaScriptExecutor to click on login button.
  JavascriptExecutor js = (JavascriptExecutor) driver;
  js.executeScript ("arguments [0].click ();" , button);

How to scroll down a page using JavaScript in Selenium?

```
JavaScriptExecutor js = (JavaScriptExecutor)driver;
js.executeScript ("window.scrollBy (0, 1000)");
```

How can we use Recovery Scenario in Selenium WebDriver?

- We can develop Recovery scenarios using exception handling i.e. by using "Try Catch Block" within your Selenium WebDriver Java tests.

How to handle Ajax calls or elements in Selenium WebDriver?

- The best approach would be to wait for the required element in a dynamic period and then continue the test execution as soon as the element is found/visible.
- This can be achieved with **WebDriverWait** in combination with **ExpectedCondition**, the best way to wait for an element dynamically, checking for the condition every second and continuing to the next command in the script as soon as the condition is met.

```
WebDriverWait wait = new WebDriverWait (driver, wait Time);
wait.until (ExpectedConditions.visibilityOfElementLocated (locator));
```

How to switch to frames in Selenium WebDriver?

- For switching between frames, use driver.switchTo ().frame ().
- First locate the frame id and define it in a WebElement.

```
WebElement fr = driver.findElementById ("theIframe");
driver.switchTo ().frame (fr);
```

What are the DesiredCapabitlies in Selenium WebDriver and their use?

- The Desired Capabilities Class helps us to tell the web driver, which environment we are going to use in our test script.
- *The set Capability* method of *the Desired Capabilities Class*, can be used in *Selenium Grid.*
- It is used to perform a *parallel execution* on *different machine configurations.*
- *It is used to set the browser properties* (Ex. Chrome, IE), Platform Name (Ex. Linux, Windows) that are used while executing the test cases.

How do you handle frames in Selenium?

We can switch to frames by following methods: –

- **By Index**

  driver.switchTo ().frame (0);

- **By Name or Id**

  driver.switchTo ().frame ("id of the element");

- **By Web Element**

  driver.switchTo ().frame (WebElement);

Give an example for method overloading concept that you have used in Selenium?

- Implicit Wait in Selenium use method overloading as we can provide different Timestamp or *Time Unit like SECONDS, MINUTES,* etc.

How do you select a value from a drop-down field and what are the different methods available?

- We can select value from drop-down using methods of Select class.
- Following are the methods:
    1. selectByVisibleText
    2. selectByValue
    3. selectByIndex

How do you capture screen-shots in Selenium and what is the best place to have the screen-shot code?

- **type cast driver object with TakeScreenshot**

```
TakesScreenshot snapshot =   (TakesScreenshot) driver;
```

- **Call getScreenshotAs method to create image file**

```
File source = snapshot.getScreenshotAs (OutputType.FILE);
```

- **Move image file to new destination**

```
File destination = new File (path);
```

- **Copy file at destination**

```
FileHandler.copy(source, destination);
```

How do you debug your automation code when it is not working as expected?

- *Add breakpoints* on the lines of code where it is not working.
- Run code in debugging mode – Use different actions like *F7 (Step Into*), *F8 (Step Over), F9 (Step Out)* to debug the problem.

How do you implement collections in your framework?

- Collections can be used in framework in situations where you have to store large number of objects.
- For example, findElements () method returns a list of all matching elements.

Tell me about the Selenium WebDriver architecture?

- WebDriver is made of following components:
    1. Language Bindings.
    2. JSON Wire Protocol.
    3. Browser Drivers.
    4. Real Browsers.
- When a test script is executed with the help of web driver, the following tasks are performed in the background.
- An HTTP request is generated and it is delivered to the browser driver for every selenium command.
- The HTTP request is received by the driver through an HTTP server.
- All the steps/instructions to be executed on the browser is decided by an HTTP server.
- The HTTP server then receives the execution status and in turn sends it back to the automation scripts.

Can I navigate back and forth in a browser using Selenium WebDriver?

- Yes. We can use Navigate method to move back and forth in a browser.

```
driver.navigate ().forward ();
driver.navigate ().back ();
```

What kind of mouse actions can be performed using Selenium?

- Following mouse actions can be performed:
  1. **doubleClick ():** Performs double click on the element.
  2. **clickAndHold():** Performs long click on the mouse without releasing it.
  3. **dragAndDrop():** Drags the element from one point and drops to another.
  4. **moveToElement():** Shifts the mouse pointer to the center of the element.
  5. **contextClick ():** Performs right-click on the mouse.

What kind of keyboard operations can be performed in Selenium?

- Following keyboard actions can be performed:
  1. **SendKeys ():** Sends a series of keys to the element
  2. **KeyUp():** Performs key release
  3. **KeyDown ():** Performs keypress without release.

What is StaleElementException? When does it occur? How do you handle it?

- This Exception occurs when driver is trying to perform action on the element which is no longer exists or not valid.
- Try to get around this by first using an explicit wait on the element to ensure the Ajax call is complete, then get a reference to the element again.

```
By byPageLoc = By.id ("element");
wait.until(ExpectedConditions.elementToBeClickable (byPageLoc));
driver.findElement(By.id("element")).click();
```

How to get the number of frames on a page?

```
List frames = driver.findElement("By.id("frame-id"));
int count = frames.getSize();
```

How to send alt or shift or control or enter or tab key in Selenium WebDriver?

- We can use sendKeys () method to send any key.

```
driver.findElement(element).sendKeys(Keys.TAB);
```

How to set the size of browser window using Selenium?

```
Dimension d = new Dimension (300,1080);
driver.manage ().window ().setSize (d);
```

What are the different types of exceptions you have faced in Selenium?

i. **NoSuchElementException:**
   - Happens when the locators are unable to find elements on the web page.
   - Ideally, the exception occurs due to the use of incorrect element locators in the findElement (By, by) method.
   - To handle this exception, use the *wait* command.
   - Use try/catch to ensure that the program flow is interrupted if the wait command doesn't help.
   - **For Example:**

```
driver.findElement (By.id ("submit")).click ();
```

   - **Exception Handling:**

```
try {
driver.findElement(By.id("submit")).click();
}
catch (NoSuchElementException e)
```

   - In this case, the exception is thrown even if the element is not loaded.
   - **Avoiding-And-Handling**: Try giving a wait command
     - The wait command below waits 10 seconds for the presence of web element with id 'submit'. Then it tries to click it. If the element is available but still the click fails, an exception is caught.

```
try
{
WebDriverWait wait = new WebDriverWait(driver,
TimeSpan.FromSeconds(10));
wait.Until(ExpectedConditions.presenceOfElementLocated(By.id("submit")));
try
{
driver.findElement(By.id("submit")).click();
}
catch (WebDriverException e)
{
System.out.println("An exceptional case.");
}
}
catch (TimeOutException e)
{
System.out.println("WebDriver couldn't locate the element");
}
```

ii. NoSuchWindowException:

- NoSuchWindowException occurs if the current list of windows is not updated. The previous window does not exist, and you can't switch to it.
- *To handle this exception, use webdriver methods called "driver.getWindowHandles ()."*
- It will throw *org.openqa.selenium.NoSuchWindowException* if the window handle doesn't exist or is not available to switch.
  - driver.switchTo ().window (handle_1);
- **Avoiding-And-Handling**: We would use window handles to get the set of active windows and then perform actions on the same.

```
for (String handle : driver.getWindowHandles()) {
try {
driver.switchTo().window(handle);
}
catch (NoSuchWindowException e) {
System.out.println("An exceptional case");
}
}
```

iii. **NoSuchFrameException:**

- When web driver is trying to switch to an invalid frame, NoSuchFrameException under NotFoundException class is thrown.
- It will throw *org.openqa.selenium.NoSuchFrameException* if a frame "frame_11" doesn't exist or is not available.
- driver.switchTo().frame("frame_11");
- Exception Handling:

```
try {
driver.switchTo().frame("frame_11");
} catch (NoSuchFrameException e)
```

- In this case, the exception is thrown even if the frame is not loaded.
- **Avoiding-And-Handling**: Try to give a wait command.

```
try {
WebDriverWait wait = new WebDriverWait(driver,
TimeSpan.FromSeconds(10));
wait.Until(ExpectedConditions.frameToBeAvaliableAndSwitchToIt(frame_11));
try {
driver.switchTo().frame("frame_11");
}
catch (WebDriverException e) {
System.out.println("An exceptional case");
}
} catch (TimeOutException e) {
System.out.println("WebDriver couldn't locate the frame");
}
```

iv. **NoAlertPresentException :**

- ***NoAlertPresentException* under *NotFoundException*** is thrown when web driver tries to switch to an alert, which is not available.
- *Org.openqa.selenium.NoAlertPresentException* will be thrown If below automation code calls accept () operation on Alert () class when an alert is not yet on the screen.
    - driver.switchTo ().alert ().accept ();

- **Exception Handling:**

```
try {
driver.switchTo ().alert ().accept ();
} catch (NoSuchAlertException e)
```

- In this case, the exception is thrown even if the alert is not loaded completely.
- **Avoiding-And-Handling**: Always use explicit or fluent wait for a particular time in all cases where an alert is expected. If the alert is available and still there is an exception, then it is caught.

```
try {
WebDriverWait wait = new WebDriverWait(driver,
TimeSpan.FromSeconds(10));
wait.Until(ExpectedConditions.alertIsPresent());
try {
driver.switchTo().alert().accept();
} catch (NoAlertPresentException e) {
System.out.println("An exceptional case");
}
} catch (TimeOutException e)
System.out.println("WebDriver couldn't locate the Alert");
}
```

v. **InvalidSelectorException:**

- This subclass of *NoSuchElementException* class occurs when a selector is incorrect or syntactically invalid. This exception occurs commonly when XPATH locator is used.
- **Example:**

  clickXPathButtonAndWait("//button[@type='button'][100]");
- This would throw an *InvalidSelectorExeption* because the XPATH syntax is incorrect.
- **Avoiding and Handling:** To avoid this, we should check the locator used because the locator is likely incorrect or the syntax is wrong. Using Firebug to find xpath can reduce this exception.

```
try {
clickXPathButtonAndWait("//button[@type='button']");
} catch (InvalidSelectorException e) {
}
```

vi. ElementNotVisibleException:

- *ElementNotVisibleException* class is a subclass of *ElementNotInteractableException* class.
- This exception is thrown when web driver tries to perform an action on an invisible web element, which cannot be interacted with.
- That is, the web element is in a hidden state.
- **For example:**
- In the below code, if the type of button with id 'submit' is 'hidden' in HTML, *org.openqa.selenium.ElementNotVisibleException* will be thrown.

```
driver.findElement(By.id("submit")).click();
Exception Handling:
try {
driver.findElement(By.id("submit")).click();
} catch (ElementNotVisibleException e)
```

- In this case, the exception is thrown even if the page has not loaded completely.
- **Avoiding-And-Handling**: There are two ways to do this. We can either use wait for the element to get completely.

```
try {
WebDriverWait wait = new WebDriverWait(driver,
TimeSpan.FromSeconds(10));
wait.Until(ExpectedConditions.visibilityOfElementLocated(By.id("submit"));
try {
driver.findElement(By.id("submit")).click();
} catch (WebDriverException e) {
System.out.println("Exceptional case");
}
} catch (TimeOutException e)
System.out.println("WebDriver couldn't find this element visible");}
```

vii. ElementNotSelectableException:

- This exception comes under *InvalidElementStateException* class.
- *ElementNotSelectableException* indicates that the web element is present in the web page but cannot be selected.
- **For example,** the below code can throw an *ElementNotSelectableException* if the id "swift" is disabled.

```
Select dropdown = new Select (driver.findElement (By.id ("swift")));
```

- **Exception Handling:**

```
try {
Select dropdown = new Select (driver.findElement (By.id("swift")));
} catch (ElementNotSelectableException e)
```

- In this case, exception is thrown even if the element becomes enabled after a while.

- **Avoiding-And-Handling**: We can add a wait command to wait until the element becomes clickable. If there is still an exception, it is caught.

```
try {
WebDriverWait wait = new WebDriverWait(driver, TimeSpan.FromSeconds(10));
wait.Until(ExpectedConditions. elementToBeClickable(By.id(”swift”));
try {
Select dropdown = new Select(driver.findElement(By.id("swift")));
} catch (WebDriverException e) {
System.out.println(“Exceptional case”);
}
} catch (TimeOutException e)
System.out.println(“WebDriver found that this element was not selectable.”);}
```

viii. **TimeoutException:**

- This exception occurs when a command completion takes more than the wait time.
- Waits are mainly used in web driver to avoid the exception *ElementNotVisibleException.*
- Sometimes test page might not load completely before next command in the program. If web driver tries to find an element in the webpage before the page completely loads, then exception *ElementNotVisibleException* is thrown.
- To avoid this exception, waits commands are added.
- However, if the components don’t load even after the wait, the exception *org.openqa.selenium.**TimeoutException*** will be thrown.

```
driver.manage ().timeouts ().implicitlyWait (10, TimeUnit.SECONDS);
driver.get (“https://www.softwaretestinghelp.com”);
```

- In the above program, an implicit wait of 10 seconds is added.
- If the page *www.softwaretestinghelp.com* doesn’t load in 10 seconds, then TimeoutException will be thrown.
- ***Avoiding and Handling****: To avoid this, we can manually check the average time for a page to load and adjust the wait*
- *Or, we can add explicit wait using JavaScript executor until the page is loaded.*
- *In the below example, JavaScript executor is used. After page navigation, we call JavaScript return document.readyState for 20 seconds until “complete” is returned.*

```
WebDriverWait wait = new WebDriverWait (driver, TimeSpan.FromSeconds(30));

wait.until(webDriver -> ((JavascriptExecutor)webDriver).executeScript("return
document.readyState").equals("complete"));

driver.get("https://www.softwaretestinghelp.com");
```

ix. NoSuchSessionException:

- This exception is thrown when a method is called after quitting the browser by WebDriver.quit();
- This can also happen due to web browser **issues like crashes** and WebDriver cannot execute any command using the driver instance.
- To see this exception, the code below can be executed.

```
driver. Quit ()
Select dropdown = new Select (driver.findElement(By.id("swift")));
```

- **Avoiding and Handling**: Always choose the latest stable version of browser to run Selenium web driver testcases.
- This exception can be reduced by using driver.quit () at the completion of all tests.
- Do not try to use them after each test case. This can lead to issues when driver instance is null and upcoming test cases try to use it without initializing.
- The below code creates WebDriver instance in the @Before Suite TestNG annotation and destroys it in @AfterSuite TestNG annotation

```
@BeforeSuite
public void setUp() throws MalformedURLException {
WebDriver driver = new FirefoxDriver();
}
@AfterSuite
public void testDown() {
driver.quit();
}
```

x. StaleElementReferenceException:

- This exception says that a web element is no longer present in the web page.
- This error is not the same as *ElementNotVisibleException.*
- *StaleElementReferenceException* is thrown when an object for a particular web element was created in the program without any problem and however; this element is no longer present in the window or web page.
- This can happen if there was a:
  - Navigation to another page
  - DOM has refreshed
  - A frame or window switch.

```
WebElement firstName = driver.findElement (By.id ("first_name"));
driver.switchTo ().window (Child Window);
element.sendKeys ("Aaron");
```

- In the code above, object firstName was created and then the window was switched. Then, WebDriver tries to type ‘Aaron’ in the form field. In this case StaleElementReferenceException is thrown.

- ***Avoiding and Handling****: Confirm that we are trying to do the action in the correct window. To avoid issues due to DOM refresh, we can use Dynamic Xpath.*

- In this case, we can use a dynamic Xpath like,

```
try {
driver.findElement (By.xpath (“//*[contains (@id, first_name’)]”)).sendKeys
(“Aaron”);
} catch (StaleElementReferenceException e)
```

- In the example above dynamic XPATH is used and if the exception is still found, it is caught.